

How to reduce technical debt...

It's all about readability and maintainable code

Presented by Christopher Ewing



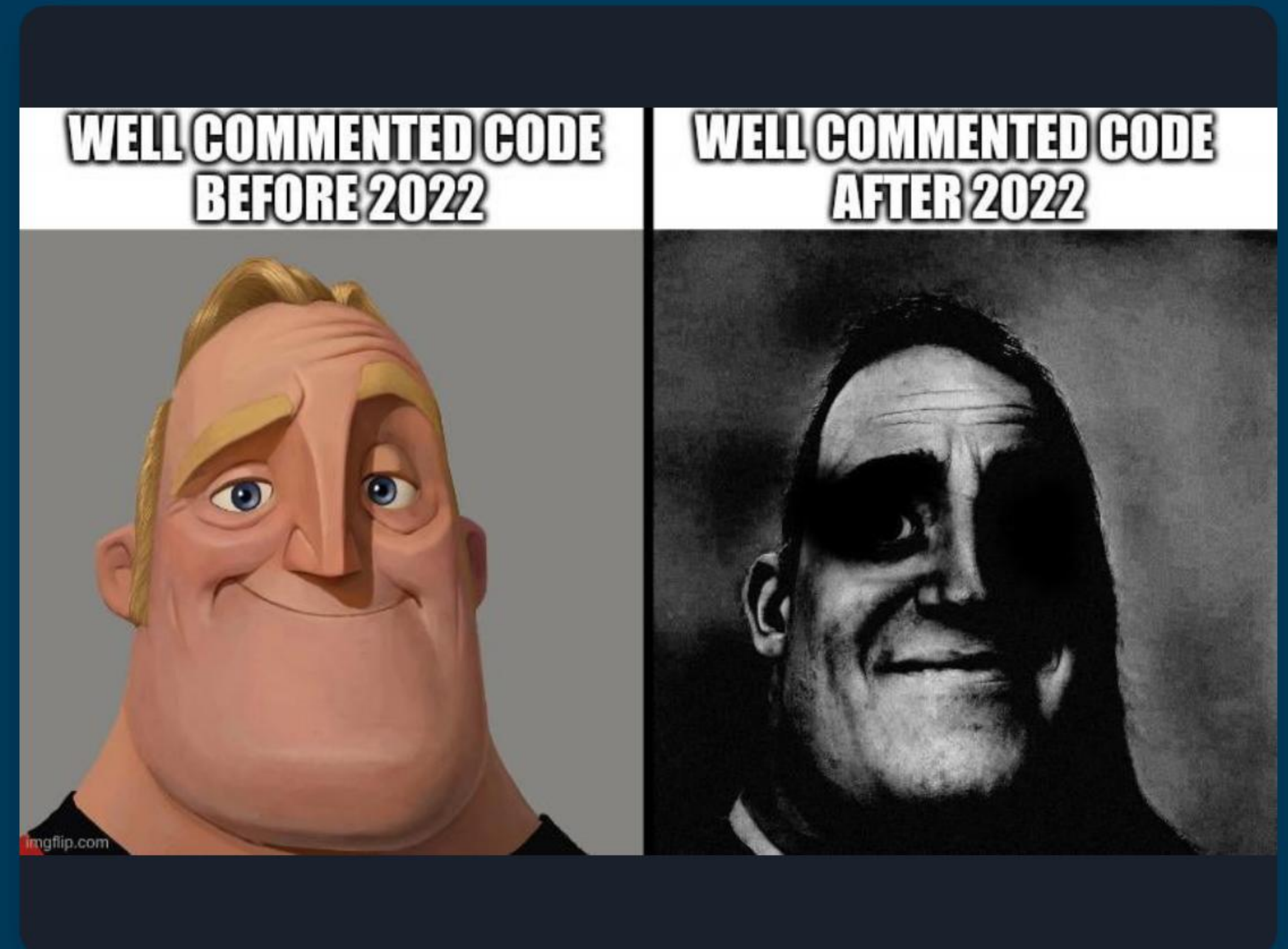
Context for this Presentation

I originally delivered this message during internal training sessions at my workplace.

While I cannot share the exact deck I presented due to confidentiality, this presentation captures the core principles and the general message of those sessions.

1. Comment and Document

- » **Clarify the complex:** Still use comments when the code logic may be difficult to follow or has non-obvious business rules.
- » **Standardize documentation:** Use docstrings appropriately for functions, classes, and modules to explain the "what" and "why".
- » **Keep them updated:** A comment that contradicts the code is worse than no comment at all.



2. Variables: Clarity is Key

Descriptive Naming

Use descriptive variable names. Name length should be proportional to scope. A short iterator like `i` is fine for three lines, but global states need explicit names.

Consider context: avoid obscure abbreviations that only you understand.

Self-Documenting Code

Prioritize clarity over brevity. Your code should be readable enough that the core functionality is obvious.

A variable named `customer_billing_address` is often much clearer than `c_b_addr`.

3. Design your program



Understand Requirements

Whiteboard the architecture and understand the core requirements before diving deep into writing code.



Testable Design

Design your structures and functions with testing in mind. Consider writing tests alongside your code to ensure reliability.



Optimize Smart

Avoid premature optimization. Focus on a readable, solid implementation first, then optimize where true bottlenecks appear.



Modernize

For conversions (e.g., SAS to Python/Pandas), reimagine the logic. Don't just translate word for word.

4. Functions: Simpler is Better

- » **Single Responsibility:** Consider breaking down long functions into shorter ones. If a function does more than one thing, it may need to be split up.
- » **Keep Indentation Shallow:** Avoid the "Pyramid of Doom" (deeply nested logic). Shallow code is generally much easier to read and maintain.
- » **Predictable Returns:** A simple function takes clear inputs and returns expected outputs without hidden side effects.



5. Avoid Hard Coding

Prefer Soft Coding

If avoidable, try not to hard code values directly into your functional logic. Magic numbers and hidden strings can make code brittle.

Aim to make your logic dynamic and adaptable to changing inputs.

Centralize Constants

When hard coding is necessary (e.g., configuration settings or mathematical constants), try to centralize them in one place or a config file.

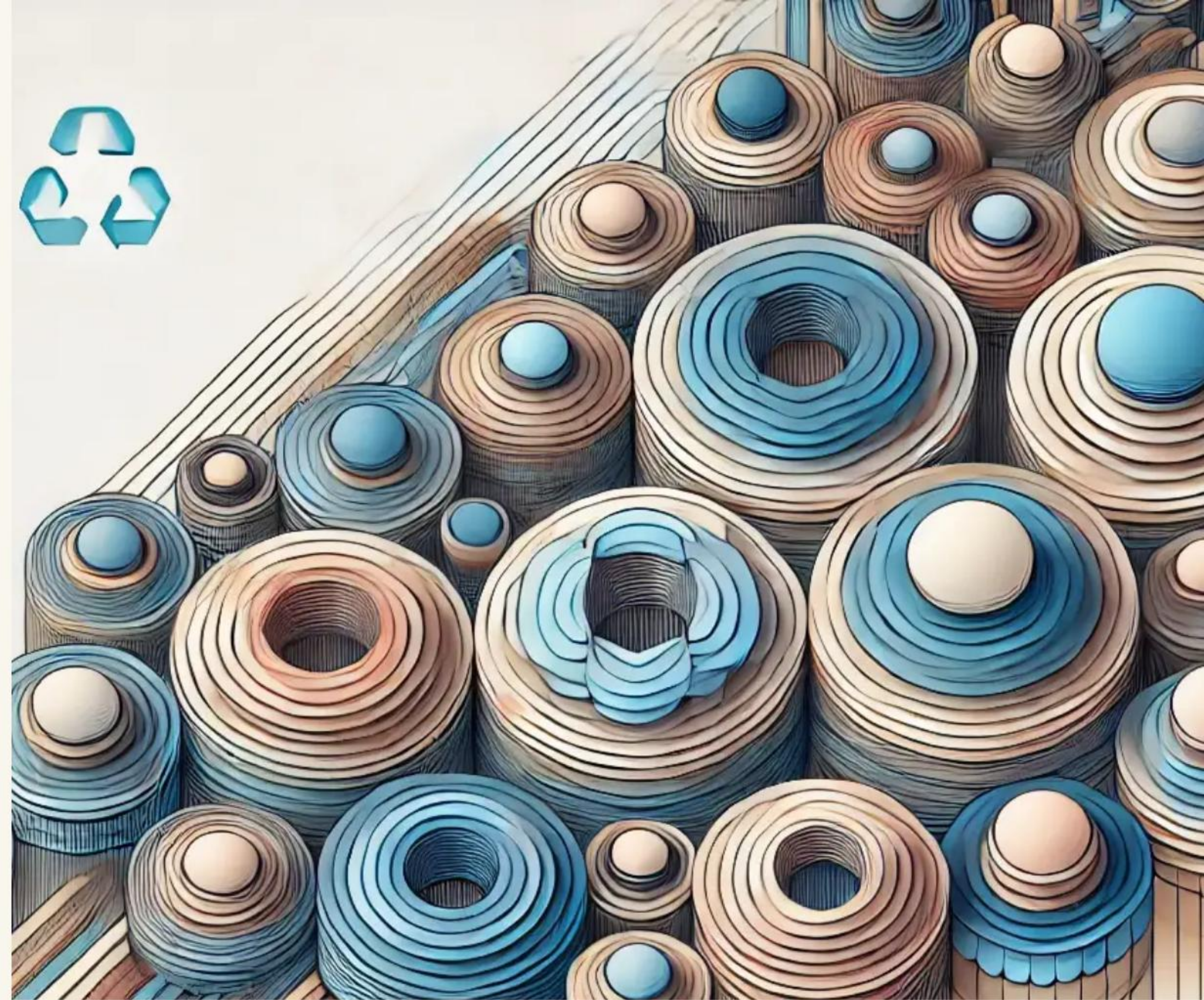
This ensures they act as a single source of truth.

6. DRY & The Rule of Three

Wait for the Rule of Three. Don't immediately abstract code the second time you write it. Wait until you use it a third time to ensure you truly understand the shared pattern.

"Duplication is far cheaper than the wrong abstraction."

— Sandi Metz



7. Leverage Version Control

Version control is a critical safety net for your codebase.

Commit often, write descriptive commit messages, and use branching strategies to protect the main production code from breaking changes.

Image Sources



<https://preview.redd.it/ifyouknowyouknow-vO-v6q8n4y77i8g1.jpeg?auto=webp&s=d646dfaf8bc6cc7c619420ec66a61f6449bf82ec>

Source: www.reddit.com



<https://preview.redd.it/minimalist-block-terrain-looking-for-thoughts-and-feedback-vO-8bx4bgpe852f1.jpg?width=640&crop=smart&auto=webp&s=88080f1269c9595a90fff4fee955ff50b9632ee2>

Source: www.reddit.com



https://miro.medium.com/1*3ViB-mMhA5XyrKVGeGximA.png

Source: vitor-azevedo.medium.com



https://png.pngtree.com/thumb_back/fh260/background/20250708/pngtree-abstract-network-background-with-soft-blue-gradient-and-white-lines-image_17603011.webp

Source: pngtree.com